

SeuGado — Ezequiel: piquetes e mapa

Desenhar os piquetes da fazenda e mostrar o estado de cada um no mapa

27/09/2026

Sua parte em uma frase: a tela em que a fazenda ganha forma. O produtor (ou nós, por ele) desenha cada piquete sobre a imagem de satélite, diz qual capim tem ali e a altura que mediu, e depois vê no mesmo mapa como está cada piquete e para onde o gado vai nesta semana.

Você entrega: backend dos piquetes (serviço e rotas) + página **Mapa** no site.

Prazo: tudo pronto e com PR aberto até domingo à noite. Segunda é integração, terça é a apresentação.

1. O SeuGado em um minuto

O pecuarista divide o pasto em **piquetes** (áreas cercadas) e move o gado de um para outro. O capim tem um ponto certo para o gado entrar (**altura de entrada**) e um ponto certo para sair (**altura de saída**). Se o gado entra tarde, o capim já passou do ponto e perdeu valor. Se sai tarde, o pasto se degrada. O SeuGado estima a altura do capim de cada piquete com satélite e clima e manda, toda semana, pelo Telegram, um plano do tipo “segunda: mova o lote Recria do Piquete 2 para o Piquete 1”.

Nada disso funciona sem saber **onde** estão os piquetes. Esse é o seu trabalho.

Palavras que você vai usar

Termo	O que é
Piquete	Uma área cercada do pasto. No sistema, um polígono no mapa com nome, área, capim e método de pastejo
Lote	Um grupo de animais que anda junto (cadastrado pelo Leandro)
Cultivar	O tipo de capim (Marandu, Mombaça...). Cada um tem suas alturas ideais
Método de pastejo	rotacionado (o gado gira entre piquetes) ou contínuo (o gado fica sempre no mesmo)
Altura medida	A altura do capim, em cm, medida com régua pelo produtor. É o ponto de partida da estimativa
Parâmetro faltante	Algumas combinações de capim x método não têm altura publicada. Nesse caso, o sistema pergunta ao produtor qual altura ele usa
Evento	Um registro imutável de algo que aconteceu (“piquete criado”, “altura medida”). O sistema nunca edita o passado: registra um evento novo
Tabelas derivadas	Tabelas reconstruídas a partir dos eventos (estado_piquete, altura_atual...). Você lê delas, mas nunca escreve nelas

Entradas e saídas: o que você recebe, de quem, e o que entrega

Esta seção é o seu contrato com o resto da equipe. **Nomes, tipos e formatos são exatamente estes**; se algo aqui parecer faltar ou estar errado, fale com o Kauê antes de inventar outro formato.

Resumo

	O quê	De quem / para quem	Como chega / sai
Recebe	desenho e dados do piquete	a sua página Mapa	POST/PUT <code>PiqueteIn</code> (JSON, seção 5.1)
Recebe	usuário logado e fazenda dele	Kauê (<code>api/auth.py</code>) e Leandro (<code>GET /me</code>)	dependências <code>Usuario</code> , <code>FazendaAutorizada</code> ; no front, <code>useFazenda()</code>
Recebe	catálogo de capins com as alturas	Kauê (<code>persistencia/catalogo.py</code>)	funções <code>carregar_catalogo</code> , <code>resolver_alturas</code> , <code>faltantes_calibracao</code>
Recebe	o plano da semana, para pintar o mapa	Leandro (rota), gerado pelo João	<code>GET /fazendas/{id}/plano/atual</code> → <code>PlanoManejo</code> (JSON) ou 404
Entrega	piquetes gravados como eventos	banco → Kauê (satélite, estado) → João (plano)	<code>piquete_criado</code> , <code>piquete_alterado</code> , <code>altura_medida</code> , <code>parametro_alterado</code>
Entrega	lista de piquetes	Leandro (formulário de lotes: “onde o lote está”)	<code>GET /fazendas/{id}/piquetes</code> → <code>Piquete[]</code>
Entrega	registrar altura medida	Leandro (página do plano, “Medições pedidas”)	POST <code>/fazendas/{id}/piquetes/{piquete_id}/alturas</code>

O Piquete que você devolve (JSON), campo a campo

Campo	Tipo	Exemplo	Origem
<code>id</code>	texto (UUID)	<code>"1111...0001"</code>	<code>estado_piquete.piquete_id</code>
<code>nome</code>	texto	<code>"Piquete 1"</code>	<code>estado_piquete.nome</code>
<code>geometria</code>	GeoJSON Polygon, <code>[longitude, latitude]</code>	<code>{"type": "Polygon", "coordinates": [[[-36.09, -9.78], ...]]}</code>	<code>ST_AsGeoJSON(estado_piquete.geometria)::json</code>
<code>area_ha</code>	número	<code>4.02</code>	<code>estado_piquete.area_ha</code>
<code>cultivar_id / cultivar_nome</code>	texto	<code>"..."</code> , <code>"Marandu"</code>	<code>estado_piquete.cultivar_id + cultivar.nome</code>
<code>metodo_pastejo</code>	<code>"rotacionado"</code> <code>"continuo"</code>	<code>"rotacionado"</code>	<code>estado_piquete.metodo_pastejo</code>
<code>situacao</code>	<code>"ocupado"</code> <code>"descansando"</code>	<code>"descansando"</code>	<code>estado_piquete.situacao</code>
<code>lote_atual_id / lote_atual_nome</code>	texto ou <code>null</code>	<code>null</code>	<code>estado_piquete.lote_atual_id + estado_lote.nome</code>

Campo	Tipo	Exemplo	Origem
ultima_altura_cm	número ou null	28.0	altura_atual.altura_cm
ultima_altura_data	texto ISO ou null	"2026-09-27"	altura_atual.data

O que você lê do plano (campos usados no mapa)

- piquetes[] (ResumoPiquete): piquete_id, situacao, lote_atual_nome, altura_hoje_cm, altura_entrada_alvo_cm, altura_saida_alvo_cm, confianca, motivo_confianca, faltantes.
- movimentacoes[] (Movimentacao): data, lote_nome, piquete_origem_id, piquete_destino_id.

PlanoManejo: o plano da semana

Tipo Python em `seugado.contratos`; em JSON via `plano_para_dict` / `plano_de_dict`; em TypeScript em `frontend/src/lib/tipos.ts`. Exemplo completo: `tests/fixtures/plano_exemplo.json`.

Campo	Tipo (Python)	Em JSON	Significado
id	UUID	texto	Id do plano
fazenda_id	UUID	texto	Fazenda dona do plano
data_geracao	datetime (UTC, com fuso)	"2026-09-28T08:00:00+00:00"	Quando foi gerado
data_inicio	date	"2026-09-28"	Primeiro dia do plano (hoje)
horizonte_dias	int	7	Sempre 7
movimentacoes	tuple[Movimentacao, ...]	lista	Ordenadas por (data, lote_nome)
alertas	tuple[Alerta, ...]	lista	Ordenados por (data, tipo, texto)
pedidos_validacao	tuple[PedidoValidacao, ...]	lista	Ordenados pelo nome do piquete
piquetes	tuple[ResumoPiquete, ...]	lista	Um por piquete ativo, em ordem de nome

Movimentacao: “mover o lote X do piquete A para o B no dia D”

Campo	Tipo	Em JSON	Significado
id	UUID	texto	Id da movimentação; é o entidade_id do evento manejo_recomendado e o que vai nos botões do bot
data	date	"2026-09-28"	Dia da movimentação (sempre dia de manejo preferido)
lote_id / lote_nome	UUID / str	texto	Lote que muda
piquete_origem_id / piquete_origem_nome	UUID None / str None	texto ou null	De onde sai
piquete_destino_id / piquete_destino_nome	UUID / str	texto	Para onde vai

Campo	Tipo	Em JSON	Significado
altura_destino_cm	float (1 casa)	número	Altura prevista do destino no início do dia
altura_entrada_alvo_cm	float	número	Altura ideal de entrada do capim do destino
altura_origem_cm	float None	número ou null	Altura prevista da origem no início do dia
altura_saida_alvo_cm	float None	número ou null	Altura de saída do capim da origem
dias_previstos	int	número	Dias que o lote deve ficar no destino
motivo	str	texto	Frase pronta, em português, para o produtor
confianca	Confianca	"alta" "media" "baixa"	Confiança da recomendação
motivo_confianca	str	texto	Frase pronta: o dado mais fraco por trás da recomendação

Alerta

Campo	Tipo	Em JSON	Significado
tipo	TipoAlerta	um de: sem_piquete_apto, capacidade_excedida, aguardando_parametro, estimativa_indisponivel, continuo_acima_maxima, continuo_abaixo_minima, lote_sem_piquete, sem_dia_de_manejo	Tipo do aviso
data	date	"2026-10-01"	Dia a que se refere
texto	str	texto	Frase pronta para o produtor
confianca / motivo_confianca	Confianca / str	texto	Como na movimentação
piquete_id / lote_id	UUID None	texto ou null	A quem se refere

PedidoValidacao: pedido de medir o capim com régua

Campo	Tipo	Em JSON	Significado
piquete_id / piquete_nome	UUID / str	texto	Piquete a medir
motivo	str	texto	Frase pronta: por que o sistema está pedindo

ResumoPiquete: a fotografia de cada piquete (usada no mapa)

Campo	Tipo	Em JSON	Significado
piquete_id / nome	UUID / str	texto	Piquete
situacao	SituacaoPiquete	"ocupado" "descansando"	Tem gado ou está descansando

Campo	Tipo	Em JSON	Significado
lote_atual_nome	str None	texto ou null	Lote que está nele
altura_hoje_cm	float None	número ou null	Altura estimada hoje; null = sem estimativa
altura_entrada_alvo_cm / altura_saida_alvo_cm	float None	número ou null	Alvos (nulos no contínuo ou sem parâmetro)
confianca / motivo_confianca	Confianca / str	texto	Confiança da estimativa do piquete
faltantes	tuple[str, ...]	lista de texto	O que falta para estimar ou planejar; vazio = tudo certo

2. Stack e onde fica o seu código

Camada	Tecnologia
Backend	Python 3.12, FastAPI, psycopg 3 (SQL puro, sem ORM), Pydantic
Banco	PostgreSQL + PostGIS no Supabase
Frontend	React + TypeScript + Vite, Leaflet + react-leaflet, Geoman (desenho de polígonos)
Login	Supabase Auth (já pronto no esqueleto)

Arquivos que são seus (só você edita):

- `src/seugado/cadastro/piquetes.py`: regras de negócio (serviço)
- `src/seugado/api/rotas_piquetes.py`: rotas HTTP (o arquivo já existe, vazio)
- `frontend/src/paginas/Mapa.tsx`: a página (já existe, com texto provisório)
- `frontend/src/componentes/mapa/*`: componentes que você criar para o mapa
- `tests/cadastro/test_piquetes.py`

Arquivos que você usa, mas não edita: `api/deps.py` (conexão), `api/auth.py` (login), `persistencia/eventos.py` (`registrar_evento`), `persistencia/projecao_db.py` (`reconstruir_projecao`), `persistencia/catalogo.py` (catálogo de capins), `frontend/src/lib/*` (`api()`, tipos e `useFazenda()`), `frontend/src/App.tsx` (a rota `/mapa` já aponta para a sua página). Se precisar de algo neles, peça ao Kauê. **Não adicione dependências novas.**

3. Preparar o ambiente

Importante: o Python não lê o `.env` sozinho. Todo comando `uv run local` leva `--env-file .env` (como nos exemplos abaixo); sem isso a API responde erro 500 e os testes de banco são pulados.

1. Instale **Git**, **Node 20+** e **uv** (gerenciador Python: `pip install uv` ou o instalador em `docs.astral.sh/uv`).
2. Clone o repositório e crie a sua branch: `feat/ezequiel-piquetes`.

3. Na raiz, copie `.env.example` para `.env` e preencha `DATABASE_URL`, `SUPABASE_URL` e `SUPABASE_ANON_KEY` (o Kauê manda em privado). **O `.env` nunca vai para o Git.**
4. Em `frontend/`, copie `.env.example` para `.env` e preencha `VITE_SUPABASE_URL` e `VITE_SUPABASE_ANON_KEY` (os mesmos valores). `VITE_API_URL` fica `http://localhost:8000`.
5. Backend: `uv sync --group dev`, depois `uv run --env-file .env uvicorn seugado.api.main:app --app-dir src --reload`. Abra `http://localhost:8000/docs`.
6. Frontend: `cd frontend && npm install && npm run dev`. Abra `http://localhost:5173`.
7. **Comece pelo backend (E1 a E3).** O site só deixa entrar na página Mapa depois que a rota `GET /me` do Leandro estiver na `main` (ele sobe primeiro, até ~10h). Quando ele avisar, dê `git pull origin main` e faça merge na sua branch. Crie a sua conta na tela de login e a sua fazenda pelo `/docs` da API (`POST /fazendas`), até a tela de onboarding ficar pronta.

Antes de cada PR: `uv run ruff check .` e `uv run --env-file .env pytest` sem erro, e `npm run build` sem erro.

4. Como gravar e ler dados (a regra do projeto)

Toda escrita segue a mesma receita, sempre dentro da mesma conexão:

```
from datetime import UTC, datetime
from uuid import uuid4
from seugado.core.models import OrigemEvento, TipoEvento
from seugado.persistencia.eventos import registrar_evento
from seugado.persistencia.projecao_db import reconstruir_projecao

registrar_evento(conn, fazenda_id, TipoEvento.PIQUETE_CRIADO, OrigemEvento.PRODUTOR,
                 ocorrido_em=datetime.now(UTC), payload={...}, ator=str(usuario.id))
# ... outros eventos, se houver
reconstruir_projecao(conn, fazenda_id) # atualiza as tabelas derivadas
conn.commit()                         # a ROTA faz o commit, uma vez, no fim
```

- `registrar_evento` valida o `payload` e recusa campo sobrando ou faltando (`ValidationError` → devolva 400).
- A conexão das rotas vem de `Conexao` (`api/deps.py`). Ela **não** faz `commit` sozinha: se você esquecer o `conn.commit()`, nada é salvo.
- Para ler, use SQL nas tabelas derivadas. Para ler como dicionário, use `conn.cursor(row_factory=dict_row)` (from `psycopg.rows` import `dict_row`).

Payloads que você vai gravar (todos os campos são obrigatórios):

```
# piquete_criado e piquete_alterado
{"entidade_id": "<uuid do piquete>", "nome": "Piquete 1", "area_ha": 4.02,
 "cultivar_id": "<uuid da cultivar>", "metodo_pastejo": "rotacionado", # ou "continuo"
 "ativo": True, "geometria_geojson": {"type": "Polygon", "coordinates": [[[lon, lat],
...]]}}
```

```
# altura_medida
{"entidade_id": "<uuid novo>", "piquete_id": "<uuid do piquete>", "data": "2026-09-27",
 "altura_cm": 28.0, "meio": "cadastro"} # "cadastro" | "web" | "bot"
```

```
# parametro_alterado (altura que o produtor usa, quando o capim não tem fonte)
{"entidade_id": "<uuid novo>", "cultivar_id": "<uuid>", "metodo_pastejo": "rotacionado",
 "campo": "altura_entrada_cm", "valor": 35.0, "origem": "produtor", "confianca": "baixa"}
```

GeoJSON usa **[longitude, latitude]**, nessa ordem. O primeiro ponto do anel se repete no fim. O Leaflet/Geoman já entrega nesse formato com `layer.toGeoJSON().geometry`.

Tabelas derivadas que você lê:

Tabela	Colunas úteis
estado_piquete	piquete_id, nome, area_ha, cultivar_id, metodo_pastejo, ativo, geometria (PostGIS), situacao ('ocupado'/'descansando'), lote_atual_id, desde, dias_descanso
estado_lote	lote_id, nome (para mostrar o nome do lote no piquete)
altura_atual	piquete_id, data, altura_cm, meio (última medição de cada piquete)
cultivar	id, slug, nome, parametros (use o catálogo em Python, abaixo)

5. Contratos da sua parte

5.1 Rotas (o frontend e você combinam exatamente isto)

Todas exigem login. As que têm `{fazenda_id}` usam `FazendaAutorizada`, que devolve 403 se a fazenda não for do usuário.

Método e caminho	Corpo	Resposta
GET /cultivares	—	Cultivar[]
GET /fazendas/{fazenda_id}/piquetes	—	Piquete[] (só ativos, por nome)
POST /fazendas/{fazenda_id}/piquetes	PiqueteIn	201 + Piquete
PUT /fazendas/{fazenda_id}/piquetes/{piquete_id}	PiqueteIn (sem altura)	Piquete
DELETE /fazendas/{fazenda_id}/piquetes/{piquete_id}	—	204; 409 se estiver ocupado
POST /fazendas/{fazenda_id}/piquetes/{piquete_id}/alturas	{"altura_cm": 28.0, "data": "2026-09-27"}	201
GET /fazendas/{fazenda_id}/parametros-pendentes	—	ParametroPendente[]
POST /fazendas/{fazenda_id}/parametros	{"cultivar_id", "metodo_pastejo", "campo", "valor"}	201

```
// PiqueteIn
{ "nome": "Piquete 1",
  "geometria": { "type": "Polygon", "coordinates": [[[-36.09,-9.78],[-36.088,-9.78],
[-36.088,-9.782],[-36.09,-9.782],[-36.09,-9.78]]]},
  "cultivar_id": "...", "metodo_pastejo": "rotacionado",
  "altura_atual_cm": 28.0, // opcional, só no POST
  "data_medicao": "2026-09-27" } // opcional; padrão: hoje
```

```
// Piquete (resposta)
{ "id": "...", "nome": "Piquete 1", "geometria": {...}, "area_ha": 4.02,
  "cultivar_id": "...", "cultivar_nome": "Marandu", "metodo_pastejo": "rotacionado",
  "situacao": "descansando", "lote_atual_id": null, "lote_atual_nome": null,
  "ultima_altura_cm": 28.0, "ultima_altura_data": "2026-09-27" }

// Cultivar
{ "id": "...", "slug": "marandu", "nome": "Marandu",
  "regimes_disponiveis": ["rotacionado", "continuo"], // métodos com altura publicada
  "calibrada": true, "faltantes_calibracao": [] }

// ParametroPendente
{ "cultivar_id": "...", "cultivar_nome": "Xaraés", "metodo_pastejo": "rotacionado",
  "faltantes": ["altura_entrada_cm", "altura_saida_cm"] }
```

Os tipos TypeScript desses objetos **já existem** em `frontend/src/lib/tipos.ts`. Use-os, sem redefinir.

5.2 O plano (para as cores do mapa)

O Leandro expõe `GET /fazendas/{fazenda_id}/plano/atual`, que devolve um `PlanoManejo` (tipo em `tipos.ts`) ou 404 se ainda não houver plano. De lá você usa:

- `piquetes[]`: um `ResumoPiquete` por piquete, com `altura_hoje_cm`, `altura_entrada_alvo_cm`, `altura_saida_alvo_cm`, `situacao`, `lote_atual_nome`, `confianca`, `motivo_confianca` e `faltantes`;
- `movimentacoes[]`: `data`, `lote_nome`, `piquete_origem_id`, `piquete_destino_id`.

Um exemplo completo está em `tests/fixtures/plano_exemplo.json`. **Use esse arquivo para desenvolver** enquanto a rota do Leandro não estiver na `main` (copie para `frontend/src/componentes/mapa/plano_exemplo.json` e troque para a rota real antes do PR).

6. Tarefas (na ordem)

E1: Catálogo de capins (`GET /cultivares`)

Por quê: o formulário do piquete precisa da lista de capins e precisa avisar quais ainda não têm estimativa no SeuGado (hoje só o Marandu tem todos os números).

Como:

1. Em `rotas_piquetes.py`, crie a rota com `Usuario` e `Conexao`. Descubra a fazenda do usuário com `fazenda_do_usuario(conn, usuario.id)` (pode ser `None`).
2. `catalogo = carregar_catalogo(conn, fazenda_id)` (de `persistencia/catalogo.py`). Ele já aplica as alturas que o produtor informou.
3. Para cada cultivar `c`: `regimes_disponiveis = [b.metodo.value for b in c.parametros_por_regime]`; `faltantes_calibracao = list(faltantes_calibracao(c, MetodoPastejo.ROTACIONADO))`; `calibrada = not faltantes_calibracao`.
4. Ordene por nome.

Critérios de aceite:

- [] Devolve as 9 cultivares; Marandu com `calibrada: true`; Mombaça com `calibrada: false` e `faltantes_calibracao` preenchido.
- [] Sem login → 401.

Fora do escopo: editar o catálogo; cadastrar capim novo.

E2: Serviço de piquetes (`cadastro/piquetes.py`)

Por quê: separar a regra de negócio da rota deixa tudo testável e mantém a rota curta.

Funções (assinaturas sugeridas):

```
def area_ha(conn, geometria: dict) -> float
def criar_piquete(conn, fazenda_id: UUID, ator: str, nome: str, geometria: dict,
                  cultivar_id: UUID, metodo: MetodoPastejo,
                  altura_atual_cm: float | None, data_medicao: date | None) -> UUID
def editar_piquete(conn, fazenda_id, ator, piquete_id, nome, geometria, cultivar_id,
metodo) -> None
def desativar_piquete(conn, fazenda_id, ator, piquete_id) -> None
def listar_piquetes(conn, fazenda_id) -> list[dict]
def registrar_altura(conn, fazenda_id, ator, piquete_id, altura_cm: float, data: date,
meio: str) -> None
def parametros_pendentes(conn, fazenda_id) -> list[dict]
def registrar_parametro(conn, fazenda_id, ator, cultivar_id, metodo, campo: str, valor:
float) -> None
```

Como:

1. `area_ha`: pergunte ao PostGIS, que calcula a área real sobre a Terra: `SELECT ST_IsValid(g), ST_Area(g::geography) / 10000.0 FROM (SELECT ST_SetSRID(ST_GeomFromGeoJSON(%s), 4326) AS g) t` com `json.dumps(geometria)`. Polígono inválido (que se cruza) ou área ≤ 0 → `ValueError`.
2. `criar_piquete`:
 - valide que o nome não se repete entre os piquetes ativos da fazenda (`estado_piquete`) e que a cultivar existe (`SELECT 1 FROM cultivar WHERE id = %s`);
 - gere `piquete_id = uuid4()`, calcule a área (arredonde para 2 casas) e registre `PIQUETE_CRIADO`;
 - se veio `altura_atual_cm`, registre também `ALTURA_MEDIDA` com `meio: "cadastro"` e `data = data_medicao or date.today()`;
 - chame `reconstruir_projecao(conn, fazenda_id)` e devolva o id. **Não faça commit aqui.**
3. `editar_piquete`: leia o piquete em `estado_piquete` (não existe → `LookupError`), recalcule a área e registre `PIQUETE_ALTERADO` com **todos** os campos e `ativo: True`.
4. `desativar_piquete`: se `lote_atual_id` não for nulo, recuse (vira 409: *“Tire o lote do piquete antes de desativá-lo”*). Senão, registre `PIQUETE_ALTERADO` com os mesmos dados e `ativo: False`.
5. `listar_piquetes`: um `SELECT` juntando `estado_piquete` (só ativo), `cultivar` (nome), `estado_lote` (nome do lote atual) e `altura_atual` (última altura). A geometria sai com `ST_AsGeoJSON(geometria)::json`. Ordene por nome.
6. `registrar_altura`: valide $0 < altura_cm \leq 400$ e `data <= hoje`; registre `ALTURA_MEDIDA` com o `meio` recebido.
7. `parametros_pendentes`: para cada par (cultivar, método) usado por piquetes ativos, rode `resolver_alturas(catalogo[cultivar_id], metodo)` (de `persistencia/catalogo.py`). Se faltantes não

for vazio, entra na lista.

8. `registrar_parametro`: o campo precisa combinar com o método. No rotacionado: `altura_entrada_cm` OU `altura_saida_cm`; no contínuo: `altura_maxima_cm` OU `altura_minima_cm`. valor entre 1 e 400. Registre `PARAMETRO_ALTERADO` com origem: "produtor" e confiança: "baixa". Isso é a ADR-014: a altura vale **só para esta fazenda** e fica marcada como informada pelo produtor.
9. Todos os eventos: `OrigemEvento.PRODUTOR`, `ocorrido_em=datetime.now(UTC)`, `ator=str(usuario.id)`.

Critérios de aceite:

- [] Criar um piquete gera 1 evento `piquete_criado` (+1 `altura_medida` se houver altura) e uma linha em `estado_piquete` com a geometria.
- [] Um quadrado de ~200 m × 200 m dá cerca de 4,0 ha.
- [] Polígono que se cruza → erro claro, nenhum evento gravado.
- [] Desativar piquete ocupado → recusado.
- [] Xaraés em rotacionado aparece em `parametros_pendentes`; depois de informar entrada e saída, some.

Fora do escopo: apagar evento; piquete com buraco ou multipolígono; importar KML/shapefile; checar sobreposição entre piquetes.

E3: Rotas (`api/rotas_piquetes.py`)

Por quê: é por elas que o site conversa com o banco.

Como:

1. Modelos Pydantic de entrada e saída iguais à seção 5.1 (`PiqueteIn`, `AlturaIn`, `ParametroIn`). Valide `metodo_pastejo` com `Literal["rotacionado", "contínuo"]`.
2. Cada rota: recebe `fazenda_id: FazendaAutorizada`, `usuario: Usuario`, `conn: Conexao` → chama o serviço → `conn.commit()` → devolve.
3. Converta erros: `ValueError` e `ValidationError` → 400 (com a mensagem); `LookupError` → 404; piquete ocupado → 409.
4. Depois de criar ou editar, devolva o piquete relido de `listar_piquetes`.

Critérios de aceite:

- [] As 8 rotas aparecem em `/docs` e funcionam por lá.
- [] Fazenda de outro usuário → 403.
- [] Nenhuma rota escreve direto em tabela derivada.

Fora do escopo: paginação; filtros; upload de arquivo.

E4: Página Mapa: desenhar e cadastrar

Por quê: o desenho dos piquetes é a única tarefa chata do onboarding (ADR-004). Tem que ser rápido e à prova de erro.

Como:

1. Layout: mapa ocupando ~70% da largura e um painel lateral. No celular, o painel fica embaixo.
2. Mapa com `react-leaflet` (`MapContainer`, `TileLayer`). Fundo de **imagem de satélite**:
`https://server.arcgisonline.com/ArcGIS/rest/services/World_Imagery/MapServer/tile/{z}/{y}/{x}`
com atribuição `Tiles © Esri`. Sem ver o pasto, ninguém desenha piquete.
3. Posição inicial: se houver piquetes, enquadre todos (`fitBounds`). Senão, centro do Brasil (`-14.2`, `-51.9`, `zoom 4`), com:
 - um campo **“Ir para coordenadas”** (aceita `-9.78`, `-36.09`, que é como o Google Maps copia);
 - um botão **“Minha localização”** (`navigator.geolocation`).
4. Desenho com Geoman: dentro do mapa, um componente com `useMap()` chama `map.pm.addControls({ position: "topleft", drawPolygon: true, drawMarker: false, drawCircleMarker: false, drawPolyline: false, drawRectangle: false, drawCircle: false, drawText: false, editMode: false, dragMode: false, cutPolygon: false, removalMode: false, rotateMode: false })`. No evento `pm:create`, pegue `e.layer.toGeoJSON().geometry`, remova a camada temporária e abra o formulário.
5. Formulário “Novo piquete”:
 - **Nome** (sugira “Piquete N+1”);
 - **Capim**, um `select` de `GET /cultivares`, com “(sem estimativa ainda)” ao lado dos não calibrados e Marandu no topo;
 - **Método**, rádio “Rotacionado” / “Contínuo”, com uma frase de ajuda em cada;
 - **Altura medida hoje (cm)**, opcional, mas com o aviso *“Sem essa medida o SeuGado não consegue estimar este piquete”*;
 - **Data da medida**, com hoje como padrão.

Salvar → `POST`. Mostre a área calculada que voltou (“4,02 ha”).
6. Piquetes salvos aparecem como polígonos (`GeoJSON` do `react-leaflet`), com o nome como rótulo permanente (`Tooltip permanent`).
7. Painel lateral: lista dos piquetes (nome, capim, método, área, situação, última altura). Clicar seleciona e centraliza. Ações: **Editar dados**, **Editar forma** (ativa `layer.pm.enable()`; ao confirmar, envia `PUT` com a nova geometria), **Registrar altura** e **Desativar** (com confirmação na própria tela, sem `window.confirm`).
8. Erros da API aparecem em uma faixa vermelha no painel, com a mensagem do backend.

Critérios de aceite:

- [] Desenhar → preencher → salvar cria o piquete, que aparece no mapa e na lista.
- [] Recarregar a página mantém tudo.
- [] Editar a forma altera a área.
- [] Funciona em tela de 360 px de largura (painel embaixo).

Fora do escopo: desenhar cercas, bebedouros ou corredores; camadas extras; modo offline.

E5: Perguntar a altura que falta

Por quê: se o produtor usa um capim sem altura publicada naquele método (ex.: Xaraés rotacionado), o piquete fica parado até ele dizer a altura que usa (ADR-014). É uma pergunta curta, e não um bloqueio.

Como: no topo do painel, se `GET /parametros-pendentes` não vier vazio, mostre um cartão por item: “O SeuGado não tem a altura de **entrada** do **Xaraés** no pastejo **rotacionado**. Qual altura você usa? [__] cm”. Cada campo faltante vira um input. Salvar → `POST /parametros` (uma chamada por campo) → recarregue as pendências.

Critérios de aceite:

- [] Criar um piquete Xaraés rotacionado faz o cartão aparecer; respondido, ele some.

Fora do escopo: editar alturas de capins que já têm fonte.

E6: O mapa mostra o plano

Por quê: é a imagem da demo: a fazenda inteira, colorida pelo estado de cada piquete, com as mudanças da semana.

Como:

1. Busque `GET /fazendas/{id}/plano/actual`. Se vier 404, mostre o mapa sem cores e o aviso “Ainda não há plano. Gere na página Plano da semana.”
2. Cor de cada piquete (use o `ResumoPiquete` com o mesmo `piquete_id`):

Condição	Cor	Legenda
faltantes não vazio ou <code>altura_hoje_cm</code> nulo	cinza	Sem estimativa
ocupado e <code>altura_hoje_cm <= altura_saida_alvo_cm</code>	vermelho	Precisa sair
ocupado	azul	Com gado
descansando e <code>altura_hoje_cm >= altura_entrada_alvo_cm</code>	verde	Pronto para entrar
descansando	amarelo	Crescendo

No contínuo, os alvos de entrada e saída são nulos: use azul (com gado) ou amarelo.

3. Rótulo do polígono: `Piquete 1 · 31 cm`. Ao passar o mouse: capim, situação, lote, confiança e `motivo_confianca`.
4. Legenda fixa no canto do mapa.
5. **Setas da semana:** para cada movimentação, desenhe uma `Polyline` tracejada do centro do piquete de origem ao centro do destino, com o rótulo `seg 28/09 · Recria`. O centro pode ser a média dos vértices ou `L.polygon(...).getBounds().getCenter()`.

Critérios de aceite:

- [] Com `plano_exemplo.json`, o Piquete 6 fica verde, o 2 fica azul, o 8 fica cinza e há 3 setas.

- [] Sem plano, a página não quebra.

Fora do escopo: gerar o plano (é do Leandro e do João); histórico de planos; animação.

E7: Testes e PR

1. `tests/cadastro/test_piquetes.py`:
 - validação de `campo` × método (sem banco);
 - com banco (pule se `SEUGADO_TEST_DATABASE_URL` não existir): criar, listar, editar, desativar.
2. `uv run ruff check .`, `uv run --env-file .env pytest`, `npm run build`.
3. Abra o PR `feat/ezequiel-piquetes` → `main` com prints do mapa. Não faça merge sozinho: o merge é na reunião de segunda.

7. Usando IA no seu fluxo

Dê à IA este documento e o arquivo que ela vai editar, e peça **uma tarefa por vez**. Exemplo:

“Leia docs/equipe/EZEQUIEL.md, seções 4, 5 e a tarefa E2. Implemente `src/seugado/cadastro/piquetes.py` exatamente com essas regras, usando `registrar_evento` e `reconstruir_projecao` já existentes. Não altere nenhum outro arquivo.”

Se a IA quiser mudar arquivo de outra pessoa, instalar pacote ou “melhorar” um contrato, a resposta é não: fale com o Kauê.

8. Dúvidas de domínio rápidas

- **Por que a altura e não a massa?** O produtor mede em centímetros com régua, e o sistema converte para quilos por dentro.
- **Por que o piquete desativado não some do banco?** Nada é apagado. Um evento marca `ativo: false`, e o histórico continua valendo para auditoria.
- **E se o produtor errar o desenho?** Ele edita a forma. Isso gera `piquete_alterado`, e a área é recalculada.